

Python (Programming Language):

Python is a [high-level](#), [general-purpose programming language](#) that emphasizes [code readability](#), simplicity, and ease-of-writing with the use of [significant indentation](#),^[38] "plain English" naming, an extensive ("batteries-included") [standard library](#), and [garbage collection](#). Python supports multiple [programming paradigms](#) but with an emphasis on [object-oriented programming](#) and [dynamic typing](#).

[Guido van Rossum](#) began working on Python in the late 1980s as a successor to the [ABC](#) programming language. Python 3.0, released in 2008, was a major revision and not completely [backward-compatible](#) with earlier versions. Beginning with Python 3.5,^[39] capabilities and keywords for typing were added to the language, allowing optional [static typing](#).^[40] As of 2026, the [Python Software Foundation](#) supports Python 3.10, 3.11, 3.12, 3.13, and 3.14, following the project's annual release cycle and five-year support policy. Python 3.15 is currently in the alpha development phase, and the stable release is expected to launch in October 2026.^[41] Earlier versions in the 3.x series have reached end-of-life and no longer receive security updates.

Python has gained extensive use in the [machine learning](#) community.^{[42][43][44][45]} It is widely taught as an introductory programming language.^[46] Since 2003, Python has consistently ranked among the top ten most popular programming languages in the TIOBE Programming Community Index, which ranks programming languages based on searches across 24 platforms.^[47]

Syntax and semantics:

Statements and control flow

Python's [statements](#) include the following:

- The [assignment](#) statement, using a single equals sign =
- The [if](#) statement, which conditionally executes a block of code, along with [else](#) and [elif](#) (a contraction of [else if](#))

- The [for](#) statement, which iterates over an *iterable* object, capturing each element to a variable for use by the attached block; the variable is not deleted when the loop finishes
- The [while](#) statement, which executes a block of code as long as boolean condition is true
- The [try](#) statement, which allows exceptions raised in its attached code block to be caught and handled by except clauses (or new syntax `except*` in Python 3.11 for exception groups); [\[92\]](#) the try statement also ensures that clean-up code in a `finally` block is always run regardless of how the block exits
- The `raise` statement, used to raise a specified exception or re-raise a caught exception
- The `class` statement, which executes a block of code and attaches its local namespace to a [class](#), for use in object-oriented programming
- The `def` statement, which defines a [function](#) or [method](#)
- The [with](#) statement, which encloses a code block within a context manager, allowing [resource-acquisition-is-initialization](#) (RAII)-like behavior and replacing a common try/finally idiom [\[93\]](#) Examples of a context include acquiring a [lock](#) before some code is run, and then releasing the lock; or opening and then closing a [file](#)
- The [break](#) statement, which exits a loop
- The `continue` statement, which skips the rest of the current iteration and continues with the next
- The `del` statement, which removes a variable—deleting the reference from the name to the value, and producing an error if the variable is referred to before it is redefined [\[9\]](#)
- The `pass` statement, serving as a [NOP](#) (i.e., no operation), which is syntactically needed to create an empty code block
- The [assert](#) statement, used in debugging to check for conditions that should apply
- The `yield` statement, which returns a value from a [generator](#) function (and also an operator); used to implement [coroutines](#)
- The `return` statement, used to return a value from a function
- The [import](#) and `from` statements, used to import modules whose functions or variables can be used in the current program. Python 3.15 adds a new functionality to lazily import with a new keyword: "The lazy keyword works with both `import` and `from ... import` statements." [\[94\]](#)
- The `match` and `case` statements, analogous to a [switch statement](#) construct, which compares an expression against one or more cases as a control-flow measure

The assignment statement (=) binds a name as a [reference](#) to a separate, dynamically allocated [object](#). Variables may subsequently be rebound at any time to any object. In Python, a variable name is a generic reference holder without a fixed [data type](#); however, it always refers to *some* object with a type. This is called [dynamic typing](#)—in contrast to [statically-typed](#) languages, where each variable may contain only a value of a certain type.

Python does not support [tail call](#) optimization or [first-class continuations](#); according to Van Rossum, the language never will. ^{[95][96]} However, better support for [coroutine](#)-like functionality is provided by extending Python's generators. ^[97] Before 2.5, generators were [lazy iterators](#); data was passed unidirectionally out of the generator. From Python 2.5 on, it is possible to pass data back into a generator function; and from version 3.3, data can be passed through multiple stack levels. ^[98]